CNENGR115 Computers and Control

Computers

Lecture 1/3 – Embedded Systems Introduction

Dr Zhongming Zhang

School Engineering

Lancaster University

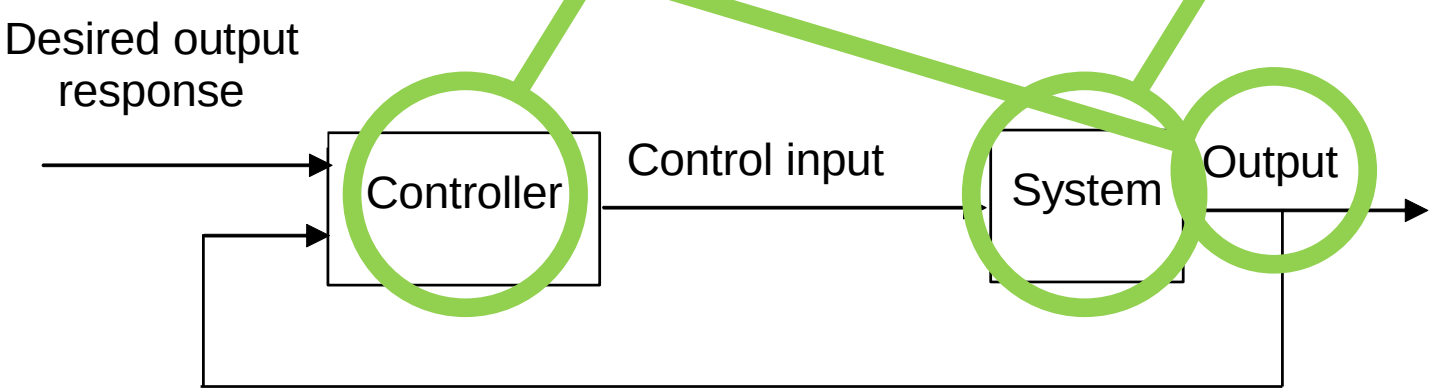
Aims:

- To introduce the concept of the embedded system as a general programmable means of responding to and controlling peripheral devices, and to develop familiarity with the architecture and programming of industry-standard devices.

Aims:

Most of us know that the microprocessor is the brain of the computer. This component is embedded in many common items such as GPS navigation, MP3 player, washing machines and countless modern devices we use every day. These devices are essentially computers which perform specific functions, also known as **Embedded System**.

Ref: C520B_Arduino_Projects_by_Chumbaka.pdf



Learning objectives

- At the end of this aspect of the CNENGR115 module you should:
  - Understand the concept of the *stored program microprocessor*.
  - Understand the *terminology* used including *hexadecimal notation*.
  - Understand the *architecture* and *limitations* of a basic *microcontroller* and how these can be overcome.
  - Be able to *write*, *test* and *execute* simple programs for a microcontroller, in C.
  - Understand the concepts of *loops* and *functions,* and understand how to use them.
  - Be able to use and set up the *principal on-chip facilities*, i.e. timers, ADCs, etc.

Recommended sources of information

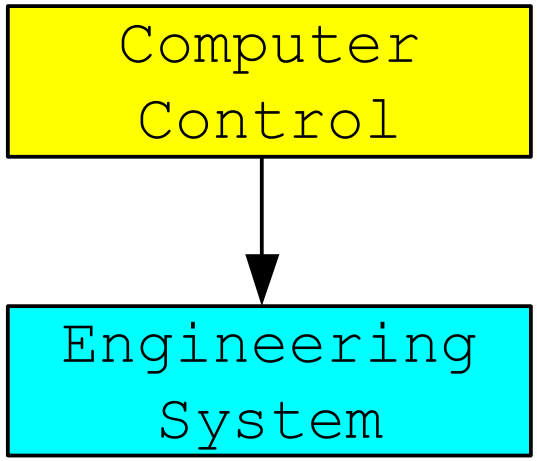
- Examples in the Arduino programming environment.
- Technical application notes.
- The Instruction Set & memory map.
- Any Arduino text.
- Various microcontroller books, dependent on device i.e. Raspberry Pi, 68000, 8051 etc.

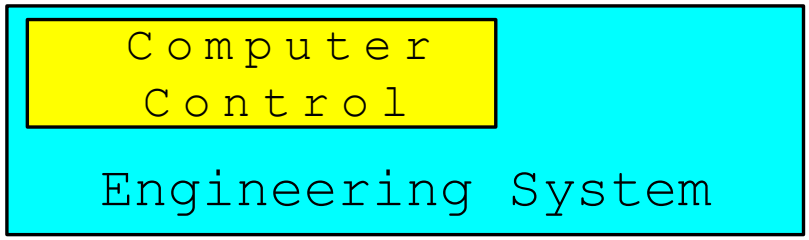
Introduction

- Microprocessors are found in PCs, calculators, palm-tops, organisers, game stations, cars, appliances, instruments, mobile phones, assembly lines.
- Often their presence is difficult to spot.
- Terms are used such as:
  - ‘embedded system’,
  - ‘embedded intelligence’.

Introduction to Embedded Systems

- If we wish to control a Mechatronic Engineering system with a computer, we have two choices:





Introduction to Embedded Systems

- If we choose to interface a PC *external* to the system then:
  - We have extensive flexibility about the scope of control, particularly with respect to the speed and memory requirements of the application.
  - However:
    - It is expensive.
    - Can be fragile.
    - Cumbersome.
    - There are many redundant facilities and capacity.
    - It is not suited to mass production.

Introduction to Embedded Systems

- If we choose to *embed* electronic intelligence inside the system then this is:
  - Cost-effective.
  - Small, low-power.
  - Reliable.
  - Robust.
  - Mass-producible.
  - Easily-serviceable.
  - But:
    - There are constraints in terms of memory capacity and speed.
    - Programming can be more complex.
    - There can often be specific electronic interfacing requirements.

Introduction to Embedded Systems

- There are a variety of approaches/protocols to embed intelligence into systems:
  - Microcontrollers.
  - Microprocessors.
  - Embedded PCs.
  - Digital signal processors.
  - Field-programmable gate arrays.
  - Neural networks.
  - Fuzzy logic.
  - Bus architectures.
- All are variations on the common theme of a dedicated, programmable electronic device carrying out instructions set up in software.

Introduction

- Microprocessors (μP) are the ‘brains’ or rather the processing agents of computers.
- They are implemented as single-chip central processing units (CPUs).
- μPs are ‘programmable hardware devices’.
- They control the other (relatively stupid) elements of the computer (disc-drives, memory, ports, input/output facilities, monitors, speakers, etc.).

History

See: http://en.wikipedia.org/wiki/List_of_Intel_microprocessors#Core_i7_2

- 1971 Intel 4004
  - » 2300 transistors
  - » 12 mm^2 silicon
  - » 640 bytes addressable RAM
  - » clock frequency of 108 kHz (10 μs)
- The *8032* version of the Philips *8051* family.
  - » 256 bytes addressable RAM
  - » Clock frequency 15 MHz (70 ns)
- 1998 Motorola HC08
  - Clock frequency 2.45 MHz (400 ns)
- 2000 Intel Itanium
  - 25.4 million transistors
  - 300 mm^2 silicon
  - 800 MHz (12 ns)
- 2012 Core i7
  - 2270 million transistors
  - 3.9 GHz....
- 2017 Core i7 – 8700K
  - 10^8 transistors / mm^2

Types of microprocessors

there is no standard!

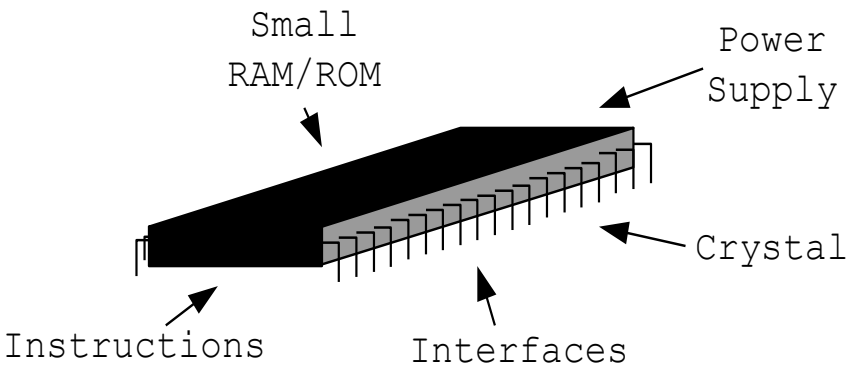
- A *microcontroller* comprises a microprocessor plus other useful circuits and facilities such as timers (counters), ADCs, memory, data transfer systems, etc.
- A *digital signal processor* (DSP) with particular features such as arithmetic functions of specific use for numerically-intensive functions (mobile phones).
- A *microprocessor* often refers to the chip controlling a computer, previously referred to then as a *microcomputer* most often now described as a PC.
- *A 'microcontroller' is a variant of embedded system, but the variety is becoming bewildering with there being many hybrid devices comprising several separate technologies in one product.*

Microcontrollers

- We will concentrate on Microcontrollers (μC) since these are most common for the control of Mechatronic systems. They are used in:
  - **Industrial control** - pumps, valves, solenoids, motors.
  - **Telecommunications** - display control, timing, encryption.
  - **Automotive** - engine management, climate control, instrumentation.
  - **Medical** - analysers, incubators, organ machines.
  - **Consumer electronics** - videos, microwaves, cookers, music systems, central heating controls.
  - **Almost everywhere now.**

Microcontrollers

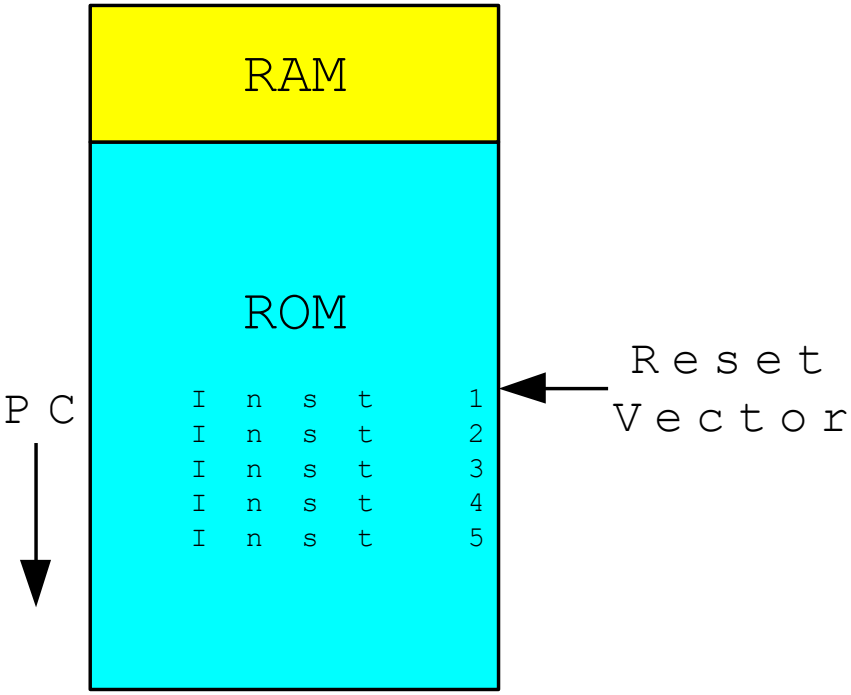
- Microcontrollers are tiny computer-based processing systems in which:
  - There is often relatively little memory, e.g. 512 bytes RAM and 32 kB ROM.
  - Ancillary features that are not present but required can be engineered separately:
    - Clock (crystal), power supply, i/o interfaces, etc.
  - They are programmed externally and then function in products independent of programming interface.
  - Speed and memory are becoming less constrained as technologies develop.



- Now the majority of devices are programmed in high-level languages, albeit based on specialized functions within associated libraries.
- Occasionally, greater efficiency and elegance can be sought via low-level languages and skillful engineering.

Programming

- µC's function by stepping through *memory* (usually ROM on the same chip) and carrying out each of the *instructions* at each successive memory location.
- The step frequency is set by the *cycle time*. This is governed by a clock crystal which can be external or on-chip. External timing can be used too.
- The *Program Counter* (PC) records the position of the controller in memory and is set by the user or, automatically, by the *Reset Vector* - this is the position to which the PC returns after the controller is reset.
- Most often this is set to the memory address at the start of ROM, since this is where the program is held.



Instruction Types

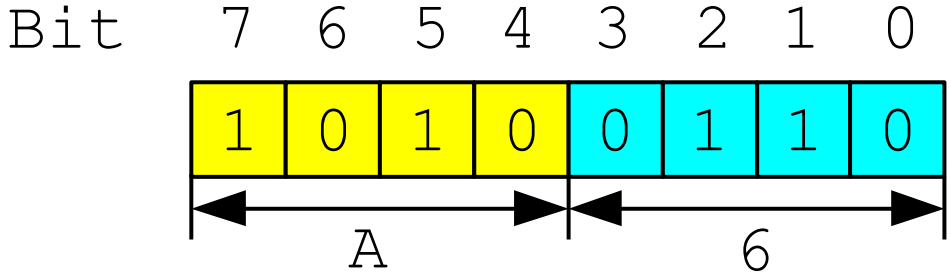
- The most important data for an Embedded System Engineer is the *Instruction Set* for the device they are using.
- All instructions either (in Assembler by way of example):
  - Move the PC to a new memory location, i.e. jump or branch to a user-defined label in code;
  - Set or clear bits in RAM:
    - to store data, i.e. *mov;*
    - to carry out mathematical functions, i.e. *add*;
    - shift the bits of a byte, i.e. *rr* or *lsr*.
- We shall concentrate on C.
  - There are often dedicated functions which use on-chip facilities implicitly, and which can be affected by changes made to those facilities.
  - The program influences the hardware status, and hence some things that are possible in software may not be possible when operated on the device.

Revision of Bits, Bytes and Binary

Bit	7	6	5	4	3	2	1	0
	0	0	1	0	0	1	1	0

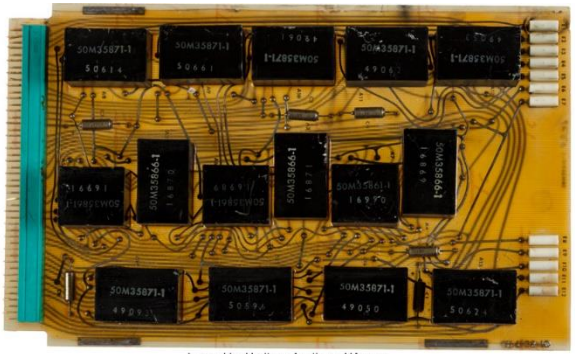
- Most memory in a MCU is arranged in 8-bit registers as shown above.
- These allow numbers between 0 and 255 to be stored in binary (base 2) by setting / clearing individual bits.
- Often, instead of representing a number, the bits will do specific jobs and a byte constitutes a set of 8 switches used to set up facilities on the chip, such as enabling ports, interrupts, etc.
- Some instructions access individual bits, others bytes.
- Some registers comprise two 8-bit registers for 16-bit capacity.
- The example 8-bit register above stores 38.

Revision of Hexadecimal



- Although μCs can cope with binary or decimal, binary is rather cumbersome and decimal becomes obscure for some addressing purposes.
- A compromise is base-16 or hexadecimal, i.e.:
  - Split a byte into two 4-bit nibbles and write the number as if these nibbles were separate;
  - Beyond '9' the letters A-F are used for 10-15;
  - The above example 'A6' is '166' in decimal.

Uses…



Imaged by Heritage Auctions, HA.com

Discrete?

Programmable?

ASIC?

Coding and plagiarism

- Academic offences can comprise:
  - Seeking online help: they send you code!?
  - Sourcing others' code, i.e. camera phones etc.
  - Blogs, chat rooms, websites.
  - There is a difference between 'researching' and 'presenting the work of others as your own'.
- Please take great care not to cross the boundary between what is yours' and what is not.